

Sweep Line Algorithm for the Union Area of Axis-Aligned Rectangles

Problem

Given n axis-aligned rectangles where each rectangle r is represented by

$$(r.\text{min}_x, r.\text{max}_x, r.\text{min}_y, r.\text{max}_y)$$

compute the total area covered by at least one rectangle.

Idea

A vertical sweep line moves from left to right across the plane. The x coordinates where rectangles start or end divide the plane into vertical strips. Within each strip the set of active rectangles does not change.

For each strip we compute the total vertical coverage and multiply it by the strip width.

Sweep Line Algorithm

Algorithm 1 UnionArea(Rectangles)

```
1: events  $\leftarrow \emptyset$ 
2: for each rectangle  $r$  do
3:   add  $(r.\text{min}_x, \text{START}, r.\text{min}_y, r.\text{max}_y)$  to events
4:   add  $(r.\text{max}_x, \text{END}, r.\text{min}_y, r.\text{max}_y)$  to events
5: end for
6: sort events by  $x$ 
7: activeIntervals  $\leftarrow \emptyset$ 
8: previousX  $\leftarrow \text{events}[0].x$ 
9: totalArea  $\leftarrow 0$ 
10: for each event  $e$  in events do
11:   currentX  $\leftarrow e.x$ 
12:   width  $\leftarrow \text{currentX} - \text{previousX}$ 
13:   coveredY  $\leftarrow \text{ComputeUnionLength}(\text{activeIntervals})$ 
14:   totalArea  $\leftarrow \text{totalArea} + \text{width} \times \text{coveredY}$ 
15:   if  $e.\text{type} = \text{START}$  then
16:     insert  $(e.\text{min}_y, e.\text{max}_y)$  into activeIntervals
17:   else
18:     remove  $(e.\text{min}_y, e.\text{max}_y)$  from activeIntervals
19:   end if
20:   previousX  $\leftarrow \text{currentX}$ 
21: end for
22: return totalArea
```

Subroutine

Algorithm 2 ComputeUnionLength(intervals)

```
1: if intervals is empty then
2:   return 0
3: end if
4: sort intervals by start coordinate
5: total  $\leftarrow$  0
6: (start, end)  $\leftarrow$  intervals[0]
7: for each interval (s, e) from the second onward do
8:   if  $s \leq end$  then
9:     end  $\leftarrow$  max(end, e)
10:  else
11:    total  $\leftarrow$  total + (end - start)
12:    start  $\leftarrow$  s
13:    end  $\leftarrow$  e
14:  end if
15: end for
16: total  $\leftarrow$  total + (end - start)
17: return total
```

Complexity

Event creation: $O(n)$ since each rectangle generates two events.

Sorting events: $O(n \log n)$ because the $2n$ events must be ordered by their x coordinate.

Sweep processing: $O(n \log n)$ since active intervals are updated and merged while scanning events.

Total time complexity: $O(n \log n)$.

Space complexity: $O(n)$ for storing events and active intervals.